

GHANA INSTITUTE OF MANAGEMENT AND PUBLIC
ADMINISTRATION

School of Technology | Department of Computer Science

CS 305: Hands-On Visual Automata Lab Manual

Interactive Machine Design & Simulation with automatonsimulator.com

Laboratory Overview & Objectives

Goal: To bridge theoretical formal definitions with visual, interactive execution. By the end of this tutorial, students will be able to:

1. Visually construct and simulate **DFAs**, **NFAs**, **Pushdown Automata (PDAs)**, and **Turing Machines (TMs)**.
2. Trace step-by-step execution states and visualize non-deterministic branching in real time.
3. Run automated batch test suites to verify language acceptance and debug rejected strings.

1. Getting Started with Automaton Simulator

Navigate to the free web-based simulator: <https://automatonsimulator.com/> (works on Chrome, Firefox, Safari, and Edge without installing any software or plugins).

1.1 Quick UI Controls & Shortcuts

Action	Mouse / Keyboard Instruction
Create State	Double-click anywhere on the blank canvas.
Set Start State	Click a state → Toggle the Start State option (arrow appears).
Set Accept State	Click a state → Toggle the Accepting State option (double circle appears).
Create Transition	Hold Shift + click and drag from state <i>A</i> to state <i>B</i> . Enter transition symbol.
Create Self-Loop	Hold Shift + click and drag from state <i>A</i> back to state <i>A</i> .
Epsilon (ε) Move	In NFAs or PDAs, leave the input field empty or enter <code>\e</code> .
Delete Element	Click the state or transition line → Press Backspace or Delete .
Move / Reposition	Click and drag any state to arrange clean layout geometry.

2. Tutorial 1: Building Your First DFA (Strings Ending in '01')

Objective: Build a DFA over alphabet $\Sigma = \{0,1\}$ accepting $L = \{w \in \{0,1\}^* \mid w \text{ ends with } 01\}$.

1. **Select Machine Type:** In the top menu, select **Finite Automaton (DFA/NFA)**.
2. **Create States:** Double-click to create three states: q_0 (Start), q_1 (Saw '0'), and q_2 (Saw '01' — Accept).
3. **Set Start & Accept:**
 - Click q_0 → check **Start**.
 - Click q_2 → check **Accepting**.
4. **Add Transitions:**
 - From q_0 : Shift-drag to q_0 with label 1; Shift-drag to q_1 with label 0.

- From q_1 : Shift-drag to q_1 with label 0; Shift-drag to q_2 with label 1.
- From q_2 : Shift-drag to q_1 with label 0; Shift-drag to q_0 with label 1.

5. Step-by-Step Simulation:

- In the bottom testing panel, enter input string: 100101.
- Click **Step Forward** (►) repeatedly. Watch the active state glow green as each character is consumed!
- Verify that the machine halts in accept state q_2 (**ACCEPTED**).

3. Tutorial 2: Visualizing NFA Non-Determinism

Objective: Build an NFA for $L = \{w \mid w \text{ contains substring } 101\}$ and observe multiple simultaneous active branches.

1. Create 4 states: q_0 (Start), q_1, q_2, q_3 (Accept).
2. Transitions:
 - At q_0 : Add self-loops on 0 and 1. Add transition to q_1 on 1.
 - At q_1 : Add transition to q_2 on 0.
 - At q_2 : Add transition to q_3 on 1.
 - At q_3 : Add self-loops on 0 and 1.
3. **Observe Non-Deterministic Parallel Simulation:**
 - Enter string 011010. Click **Step Forward**.
 - Notice that when the first 1 is read, the simulator lights up **both** q_0 and q_1 simultaneously!
 - If a branch reaches a dead end, it disappears. If at least one branch lands on q_3 , the string is accepted.

4. Tutorial 3: Simulating Pushdown Automata (PDAs) with Stack Memory

Objective: Build a PDA for the non-regular language $L = \{0^n 1^n \mid n \geq 0\}$.

1. **Switch Mode:** In the top bar, set Machine Type to **Pushdown Automaton (PDA)**.
2. **Stack Syntax in the Simulator:** Transitions are labeled as input, pop $\rightarrow$ push.
 - 0, $\epsilon \rightarrow 0 \implies$ Read input 0, pop nothing (ϵ), push 0 onto stack.
 - 1, 0 $\rightarrow \epsilon \implies$ Read input 1, pop 0 from stack, push nothing (ϵ).
 - ϵ , \$ $\rightarrow \epsilon \implies$ Read empty input, pop stack bottom marker \$, Accept!
3. **Watch the Stack Pane:** During simulation, watch the vertical stack visualization dynamically grow and shrink as symbols are pushed and popped in LIFO order!

5. Tutorial 4: Simulating Turing Machines (Tape Visualization)

Objective: Trace a Turing Machine deciding $L = \{0^n 1^n 0^n \mid n \geq 0\}$.

1. Set Machine Type to **Turing Machine**.
2. **Tape Transition Syntax:** read $\rightarrow$ write, direction (e.g., 0 $\rightarrow$ X, R or 1 $\rightarrow$ Y, L).
3. **Tape Animation:** Watch the virtual read/write head traverse the infinite tape cells, modifying symbols and reversing directions!

6. Graded Laboratory Assignment (Due in Lab Session)

Student Submission Tasks

1. Build a DFA on <https://automatonsimulator.com/> that accepts all binary strings containing an **odd number of 1's and an even number of 0's**.
2. Use the **Export** → **PNG / SVG** feature to export your state diagram.
3. In the **Test Suite Panel**, run batch tests on strings: 10, 010, 1100, 10011. Take a screenshot of the batch test table results.
4. Submit the PNG diagram and test table screenshot to the GIMPA Learning Management System.